

DSAII- HW4

Tomas Salaz

March 2, 2026

Problem 1

Prove that a Tree, T with n nodes has $n - 1$ edges. Proof by induction on n , the number of nodes.

BC: $n = 1$ is the base case. This case is trivially true since a single node has no edges, this $1 - 1 = 0$, which is true.

IH: Every tree with fewer than n nodes has exactly $n - 1$ edges.

IS: Given a Tree T with n nodes, consider the longest simple path in T . Let v be the last node in this path, v cannot connect to any new node since trees are acyclic. Therefore, v only has one predecessor, its parent.

If we remove this node, v , we get a Tree T' that has $n - 1$ nodes. T' is still a valid tree by definition (acyclic and connected) and By IH, this tree has exactly $n - 2$ edges. Since T has exactly one more edge than T' , the number of edges for T is given by $(n - 2) + 1 = n - 1$ edges. Thus every Tree with n nodes has $n - 1$ edges.

Problem 2

Prove that with bags that are 3, 5, and 8 kilos, we can fill any order of any n amount of kilos, given $n \geq 8$ and unlimited bags.

BC: For the base cases we must consider $n = 8, 9, 10$ since the smallest bag is size 3 and we can construct sums once we have these 3 cases.

Case 1: $n = 8 \rightarrow 8$

Case 2: $n = 9 \rightarrow 3 + 3 + 3$

Case 3: $n = 10 \rightarrow 5 + 5$

Thus all base cases are trivially proven.

IH: For every k where $8 \leq k < n$, an order of k kilos can be filled

IS: We must prove for $n \geq 11$. Since $n \geq 11 = n - 3 \geq 8$.

By the IH, $8 \leq n - 3 < n$, thus the IH applies and inductively proves that there is some combination of bags that can fill the orders.

Problem 3

$f(n) = \min \#$ of bags to fill n orders (if possible) or ∞ if an order cannot be filled. **Part A**

For the recurrence relation, we can use $f(0) = 0$ as a base case since this is the minimum number of bags that we need to fill.

From Problem 2, we know that $(n = 1, 2, 4, 7)$ are impossible cases, so we can define those in the base case with ∞ .

The last part is the rest of the cases, and we seen from problem 2 we can use the sizes of each bag as a 'filler', in this case, we just take the minimum. All together the recurrence is

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ \infty & \text{if } n < 0 \\ \min[f(n-3), f(n-5), f(n-8)] + 1 & \text{if } n > 0 \end{cases}$$

Part B

The array is simple, and is described above. All the impossible values get ∞ and the rest are simple if we use basic algebra.

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
f(n)	0	∞	∞	1	∞	1	2	∞	1	3	2	2	4	2	3	3	2

Part C

The Algorithm is below

```
compute_f(n) {
    f[0] = 0;
    for i from 1 to n {
        f[i] = -1 //Like infinty for impossible cases

        if (i >= 3 and f[i-3] != -1)
            f[i] = min(f[i], f[i-3] + 1)

        if (i >= 5 and f[i-5] != -1)
            f[i] = min(f[i], f[i-5] + 1)

        if {i >= 8 and f[i-8] != -1)
            f[i] = min(f[i], f[i-8] + 1)
    }
    return f[n]
}
```

The structure of the algorithm makes sure that each call does the work for a given n exactly one time, so there is no overlap.

Overall there are $n + 1$ entries we fill in the array, and there are at most 3 comparisons per entry.

So all together, the runtime in the worse case is

$$n + 1 \text{ entries} \cdot O(1) \text{ work per entry} = O(n)$$

To augment the algorithm to keep track of the amounts of bags needed, we can use another array called **choices**[i] where we keep track of the bags needed for n orders, so

choices[i] = whichever of 3,5,8 used for $\min(f[i])$

The algorithm would produce the array With this table, we can see that $f(13) =$

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
f(n)	0	∞	∞	1	∞	1	2	∞	1	3	2	2	4	2	3	3	2
choice	-	-	-	< 3	-	< 5	< 3	-	< 8	< 3	< 5	< 3	< 3	< 5	< 3	< 5	< 8

2 where we choose 5. $13-5=8$ so we go back to entry 8, thus the 2 bags are 5 and 8.

Another example is for $f(9) = 3$, we simply choose the 3 kilo bag 3 times.

Problem 4

Part A

For the function on, we can clearly see the base cases are $n = 0, 1$, and $f(0) = 2, f(1) = 5$

The Recurrence part has $2f(n-1) - f(n-2)$ for $n \geq 2$

This can be solved using the Annihilator Method.

$$T = \langle T_n \rangle$$

$$LT = \langle T_{n-1} \rangle$$

$$L^2T = \langle T_{n-2} \rangle$$

$$L^2T = 2T_{n-1} - T_{n-2}$$

$$L^2T - 2T + L = \langle 0 \rangle = (L - 1)^2$$

Using the initial conditions, $f(0) = 2$ and $f(1) = 5$, we can use the Lookup Table to solve get the form and solve for the constants.

$$f(n) = c_1 + c_2n$$

$$f(0) = 2 = c_1 + c_2(0) \rightarrow c_1 = 2$$

$$f(1) = 5 = 2 + c_2n \rightarrow c_2 = 3$$

So the recurrence is $f(n) = 3n + 2$

Checking with $n = 0..3$

$$f(0) = 3(0) + 2 = 2$$

$$f(1) = 3(1) + 2 = 5$$

$$f(2) = 3(2) + 2 = 8 \iff f(2) = 2f(1) - f(0) = 2(5) - 2 = 8$$

$$f(3) = 3(3) + 2 = 11 \iff f(3) = 2f(2) - f(1) = 2(8) - 5 = 11$$

Part B

For the runtime of the recurrence, every call to $f(n)$ in the algorithm does

2 recursive calls and a constant amount of computations. We can model the runtime using $T(n)$, where the base cases runtimes are both $O(1)$, so

$$T(0) = T(1) = O(1)$$

For the other cases we use the 2 recursive call plus the constant for computation.

$$T(n) = T(n-1) + T(n-2) + O(1) \text{ for } n \geq 2$$

This is the same as the Fibonacci Recurrence from class, so the answer for this part is

$$T(n) = O(\Phi^n) \text{ where } \Phi = \frac{1+\sqrt{5}}{2}$$